

For bkhack, the user gets to familiarize with the concept of *stream-based programming*. Indeed, visitors of the site will often catch glance of command hints and tips as they are littered about the user interface. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat.

Stream programming in bkhack is sh pipelining [She], with certain flavorful differences. The most important difference is that, instead of dealing with textual data and lines and files, here we deal with abstract post data and post counts. Consider

```
feed | split -c 10
```

where `split` has the option `-c NUM` where `NUM` is the count size of each chunk. Conventionally, the POSIX `split` command accepts the option `-l NUM` where `NUM` is line number size of chunk.

Another difference is that there is no filesystem, no concept of space. There is a command for every page, but there's no way to navigate "relatively" such as going "back" or "forth"; in other words, there's no `cd` command.

0.1. Shell and kernel

The original sh language is a user interface for the system shell, which itself is an interface for the kernel, as part of the shell-kernel architecture [AT&]. In comparison, the bkhack shell language is highly abstract and doesn't need a stand-in kernel analogy: at most, the bkhack website's command system attempts to emulate the shell-kernel architecture, so that the user feels the shell-kernel architecture even when it's not really there.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

0.2. Pipeline

Introduced by Douglas McIlroy, pipelining enables commands to compose with each other parsibly simply via textual data. In this composition, there are relationships between commands to form the pipeline by parts. This composition is part of the design patterns in designing Unix programs as a whole [Ray03].

The *source* is the start of the pipeline, it exclusively produces output. The *sink* is the end of the pipeline, it exclusively consumes the input. In-between are *filters*, where data are passed in-and-out and get transformed in the process¹. So, a simple pipeline is one with a structure source-filter-sink. A complex pipeline is one that can "branch"; the way to branch is to organize each pipeline branch as each group of commands as in §0.3, then use special filters to branch on conditions. The rest are *cantrips*—commands that don't produce output but instead apply an effect onto the current environment.

It's worth noting that, even on the conceptual level, the evaluation order of the part commands is that all commands start simultaneously when a pipeline runs.

For the bkhack shell language, pipelining is first-class citizen. The grammar expects all commands to unequivocally form a pipeline, as evident by Table 1, and multiple groups of commands will connect to form branching pipelines.

0.3. Function

A reusable pipeline or grouping of commands would be called a *function*. In the bkhack shell language, Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequae doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificare non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos.

¹in the context of a pipeline-based system architecture, these are called *middle-wares*

<code><program></code>	<code>::= <command> (<program>)?</code>	<i>Simple program</i>
	<code> { <program> <end of seq> } (<program>)?</code>	<i>Composite program</i>
	<code> (<program>) (<program>)?</code>	<i>Subshell</i>
<code><command></code>	<code>::= <word> (<command>)?</code>	
<code><end of seq></code>	<code>::= ;</code>	
<code><word></code>	<code>::= Omitted</code>	
<code><seq connective></code>	<code>::= ;</code>	<i>Default continuation</i>
	<code> &&</code>	<i>Success continuation</i>

Table 1: Grammar for the bkhack shell language

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos.. Function is a useful abstraction. For example, since `feed` can be treated as a function simply built-in, it's possible to customize the behavior of `feed` by overloading it. Indeed, the default `feed` command in bkhack, which automatically has limiting of 15 items pagination, is simply a function

```
feed() { feed | split -c 15 | cut -f1 |
sort --hot ;}
```

which the user can customize. The rest of bkhack shell commands are exposed like so, thus the settings system.

0.4. Grammar

The grammar of the bkhack shell language, given in EBNF form in Table 1, Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri. Notice how the command rule do not distinguish

parts into flags or values which are expected from most real-world usage. Indeed, the responsibility of interpreting these parts, a.k.a. *options*, is left to the higher-level *option parser*, such as POSIX `getopts`.

0.5. Typing

The typing of the bkhack shell language, given in Figure 1, Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri. This is useful when shell commands have to be embedded in a statically-typed programming language.

0.6. Reference

- [She] “Shell Command Language.” IEEE, The Open Group. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/utilities/V3_chap02.html
- [AT&] “AT&T Archives: The UNIX Operating System.” [Online]. Available: <https://youtu.be/tc4ROCjYbm0?t=296>
- [Ray03] E. S. Raymond, “Interfaces–Unix Interface Design Patterns,” in *The Art of Unix Programming*, 2003, ch. 11, pp. 302–320.

$$\begin{array}{c}
\frac{}{\text{nil } () : \text{Program } T} \text{Zero} \\
\\
\frac{t : \text{Program } A}{\text{\%}(t) : \text{Cmd } A} \text{Subsection} \\
\\
\frac{x : \text{Cmd } T \quad x' : \text{Program } T}{x \parallel \% x' : \text{Program } T} \text{Cons} \\
\\
\frac{t : \text{Program } A}{\text{obs}(t) : A} \text{Observe}
\end{array}$$

Figure 1: Typing for the bkhack shell language